Chapter 3. Introduction to Software Security Touchpoints[1]

[1] Small portions of this chapter appeared in original form in Software Development magazine in September 2005 under the title "The 7 Touchpoints of Secure Software" [McGraw 2005].

Touchpoints, which are universal, are those predictable times that occur just before a surge of rapid growth in any line of development—motor, cognitive, or emotional. ...

—T. Berry Brazelton

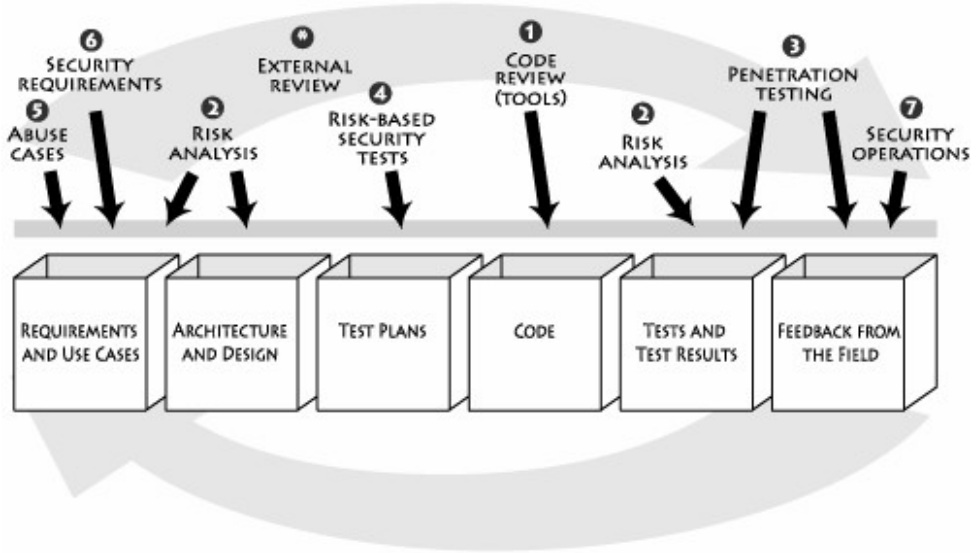
A key aim of this book is to explore and describe a set of software security best practices that I call touchpoints. Putting software security into practice requires making some changes to the way organizations build software. The good news is that these changes do not need to be fundamental, earth shattering, or cost prohibitive. In fact, adopting a straightforward set of engineering best practices, designed in such a way that security can be interleaved into existing development processes, is often all it takes. Integrating software security best practices into the software development lifecycle is the center of the three pillars of software security.

The software security best practices that I prescribe have their basis in good software engineering and involve explicitly pondering the security situation throughout the software lifecycle. This means knowing and understanding common risks, designing for security, and subjecting all software artifacts to thorough, objective risk analyses and testing. During these activities, software risk should be explicitly tracked and monitored according to the RMF presented in Chapter 2. This chapter presents a quick introduction to the software security touchpoints (a 50,000-foot view, really) and suggests an ordering for their adoption.

Figure 3-1, which also adorns the inside front cover of this book, specifies the software security touchpoints and shows how software practitioners can apply them to the various software artifacts produced during software development. This means understanding how to work security engineering into requirements, architecture, design, coding, testing, validation, measurement, and maintenance.

Figure 3-1. Lightweight software security best practices called touchpoints are applied to various software artifacts. The best practices are numbered according to effectiveness and importance. Note that by referring only to software artifacts, we can avoid battles over any particular process.

[View full size image]



Although the artifacts are laid out according to something that looks like a traditional waterfall model in the picture, most organizations follow an iterative approach today, which means that touchpoints will be cycled through more than once as the software evolves. In any event, by focusing on the artifacts we can avoid broader process issues (including the ever-present warfare surrounding which software process is the one true way and the light).

As I discuss in Chapter 1, the software security touchpoints are designed to be process agnostic. That is, the touchpoints can be applied no matter which software process you use to build your software. As long as you are producing some minimal set of software artifacts (and every project should at least be producing code!), you can apply the touchpoints.

I used to present the software security touchpoints in order from left to right. Although that works OK, a better pedagogical approach is to order the touchpoints by their natural utility and present them in some sort of ranking. Some touchpoints are by their very nature more powerful than others, and you should adopt the most powerful ones first.

Here are the touchpoints, in order of effectiveness:

1. Code review
2. Architectural risk analysis
3. Penetration testing
4. Risk-based security tests
5. Abuse cases
6. Security requirements
7. Security operations

The ordering I describe will not be a perfect fit for every organization. In fact, the ordering reflects a bias developed over many years of applying these practices in code-o-centric organizations. For that reason, code review comes before architectural risk analysis. However, the fact is that both of the top two touchpoints are critical. If you do code review and skip architectural risk analysis, you will not properly address the software security problem. Harking back to my definitions in Chapter 1, software defects that lead to security problems come in two varieties: bugs and flaws.

Code review aims at finding the bugs. Architectural risk analysis aims at finding the flaws. If you skip one or the other, you're most likely to solve only half the problem. (Remember the 50/50 bug/flaw split.) In any event, the top two touchpoints can be swapped around without any loss of generality.

As for the rest of the touchpoints, the ranking I present is based on years of experience applying the touchpoints at many different kinds of organizations, ranging from large independent software vendors to huge credit card consortiums. The ordering is not absolute. However, any attempt to change the order, say, by doing penetration testing before you do code review, is likely to be not as successful as the way I suggest. Ironically, the "penetration testing first" ordering is the ordering found in most organizations dealing with software security today, especially those shops where the security division is pushing software and application security. This ordering reflects the reactive approach to security that I am trying to counter by talking about building security in and by involving actual builders in the process.

Big organizations can adopt several touchpoints simultaneously in some cases. For more on adopting touchpoints in a large enterprise, see Chapter 10

Flyover: Seven Terrific Touchpoints

1. Code Review (Tools)

Artifact: Code

Example of risks found: Buffer overflow on line 42

All software projects produce at least one artifact—code. This fact moves code review to the number one slot on our list. At the code level, the focus is on implementation bugs, especially those that static analysis tools that scan source code for common vulnerabilities can discover. A taxonomy of these bugs can be found in Chapter 12. Several tools vendors now address this space. Code review is a necessary but not sufficient practice for achieving secure software. Security bugs (especially in C and C++) are a real problem, but architectural flaws are just as big a problem. In Chapter 4 you'll learn how to review code with static analysis tools.

Doing code review alone is an extremely useful activity, but given that this kind of review can only identify bugs, the best a code review can uncover is around 50% of the

security problems. Architectural problems are very difficult (and mostly impossible) to find by staring at code. This is especially true for modern systems made of hundreds of thousands of lines of code. A comprehensive approach to software security involves holistically combining both code review and architectural analysis.

2. Architectural Risk Analysis

Artifact: Design and specification

Examples of risks found: Poor compartmentalization and protection of critical data; failure of a Web Service to authenticate calling code and its user and to make access control decisions based on proper context

At the design and architecture level, a system must be coherent and present a unified security front. Designers, architects, and analysts should clearly document assumptions and identify possible attacks. At both the specifications-based architecture stage and at the class-hierarchy design stage, architectural risk analysis is a necessity. At this point, security analysts uncover and rank architectural flaws so that mitigation can begin. Disregarding risk analysis at this level will lead to costly problems down the road.

Note that risks crop up during all stages of the software lifecycle, so a constant risk management thread, with recurring risk-tracking and monitoring activities, is highly recommended. Chapter 2 describes the RMF process and how to apply it. Chapter 5 teaches about architectural risk analysis and will help you ferret out flaws in software architecture.

3. Penetration Testing

Artifact: System in its environment

Example of risks found: Poor handling of program state in Web interface

Penetration testing is extremely useful, especially if an architectural risk analysis informs the tests. The advantage of penetration testing is that it gives a good understanding of fielded software in its real environment. However, any such testing that doesn't take the software architecture into account probably won't uncover anything interesting about software risk. Software that fails during the kind of canned black box testing practiced by prefab application security testing tools is truly bad. Thus, passing a low-octane penetration test reveals little about your actual security posture, but failing a canned penetration test indicates that you're in very deep trouble indeed (see Chapter 1).

One pitfall with penetration testing involves who does it. Be very wary of "reformed hackers" whose only claim to being reformed is some kind of self-description.[2] Also be aware that network penetration tests are not the same as application or software-faced penetration tests. If you want to do penetration testing properly, see Chapter 6.

[2] How do we know they're reformed? Because they told us they were reformed.

4. Risk-Based Security Testing

Artifact: Units and system

Example of risks found: Extent of data leakage possible by leveraging data protection risk

Security testing must encompass two strategies: (1) testing of security functionality with standard functional testing techniques and (2) risk-based security testing based on attack patterns, risk analysis results, and abuse cases. A good security test plan embraces both strategies. Security problems aren't always apparent, even when you probe a system directly, so standard-issue quality assurance is unlikely to uncover all critical security issues. QA is about making sure good things happen. Security testing is about making sure bad things don't happen. Thinking like an attacker is essential. Guiding security testing with knowledge of software architecture, common attacks, and the attacker's mindset is thus extremely important. Chapter 7 shows you how to carry out security testing given some insight into the system's construction.

5. Abuse Cases

Artifact: Requirements and use cases

Example of risks found: Susceptibility to well-known tampering attack

Building abuse cases is a great way to get into the mind of the attacker. Similar to use cases, abuse cases describe the system's behavior under attack; building abuse cases requires explicit coverage of what should be protected, from whom, and for how long. Underused but important, abuse and misuse cases are the subject of Chapter 8. Practitioners wondering how abuse cases might work for them will get lots of mileage out of that chapter.

6. Security Requirements

Artifact: Requirements

Example of risks found: No explicit description of data protection needs

Security must be explicitly worked into the requirements level. Good security requirements cover both overt functional security (say, the use of applied cryptography) and emergent characteristics (best captured by abuse cases and attack patterns). The art of identifying and maintaining security requirements is a complex undertaking that deserves broad treatment. Interested readers are encouraged to check out the references in the Security Requirements box on the next page for pointers. A brief treatment of the subject can be found spread throughout Chapters 7 and 8.

7. Security Operations

Artifact: Fielded system

Example of risks found: Insufficient logging to prosecute a known attacker

Software security can benefit greatly from network security. Well-integrated security operations allow and encourage network security professionals to get involved in applying the touchpoints, providing experience and security wisdom that might otherwise be missing from the development team. Battle-scarred operations people carefully set up and monitor fielded systems during use to enhance the security posture. Attacks do happen, regardless of the strength of design and implementation, so understanding software behavior that leads to successful attack is an essential defensive technique. Knowledge gained by understanding attacks and exploits should be cycled back into software development.

*. External Analysis

This is not really a touchpoint, but it's important enough to emphasize so I've put it in the touchpoints picture anyway. External analysis (i.e., analysis by somebody outside the design team) is often a necessity when it comes to security. All software security touchpoints are best applied by people not involved in the original design and implementation of the system.

Every programmer has been stuck for hours working on a bug only to have a buddy (coming to drag you off for pizza) show up and point out the error: "How come you did that?!" This always warrants a huge groan. Argh! This phenomenon can happen in all stages of the software lifecycle—one reason why external analysis is a necessity.

Security Requirements

Study of security requirements is fairly new, and the literature is spotty. Here are two pointers to recommended reading:

Jonathan D. Moffett, Charles B. Haley, and Bashar Nuseibeh, "Core Security Requirements Artifacts," Technical Report 2004/23. Department of Computing, The Open University, Milton Keynes, UK, June 2004.

Nancy Mead, "Requirements Engineering for Survivable Systems," Technical Report CMU/SEI-2003-TN-013. Pittsburgh, PA: Software Engineering Institute, Carnegie Mellon University, 2003.

Why Only Seven?

Some approaches to software security are way too bulky for most organizations to swallow. By limiting the touchpoints to seven best practices, I hope to make effective best practices easier to adopt while still making a huge impact on software security. The touchpoints are not only amenable to whatever process you already follow to make software (you do ship software already, right?) but also lightweight and easy to use. If you apply the seven terrific touchpoints outlined here, your software will be much more secure.

Black and White: Two Threads Inextricably Intertwined

As I note in the Preface, the two threads of black hat and white hat activities intertwine to make up software security. This idea serves as inspiration for the cover of this book. The yin/yang design is the classic Eastern symbol related to the inextricable mixing of standard Western polemics. Eastern philosophies are for this reason called holistic. A holistic approach, mixing yin and yang—that is, mixing the black hat and white hat approaches—is just what the doctor ordered.

I define destructive activities as those about attacks, exploits, and breaking software. These kinds of things are represented by the black hat. I define constructive activities as those about design, defense, and functionality. These are represented by the white hat. Perhaps a less judgmental way to think about the dichotomy is in terms of defense and offense. Neither defense nor offense is intrinsically bad or good, and both are necessary to play almost any sport well. In any case, based on destroying and constructing, we can look back over the touchpoints and describe how the black and white threads intertwine.

Code review is a white hat (constructive) activity informed by a black hat history. The idea is to avoid implementation problems while we build software to be secure.

Architectural risk analysis is a white hat (constructive) activity also informed by a black hat history. In this case, we work to avoid design flaws while we build software to be secure.

Penetration testing is a black hat (destructive) activity. The best kind of penetration testing is informed by white hat knowledge of design and risk. But all the penetration testing in the world will not build you secure software.

Risk-based security testing is a mix of constructive and destructive activities that requires a holistic two-hat approach. Because risk-based security testing is driven by abuse cases and risk analysis results as well as functional security requirements, a mix of black hat and white hat is unavoidable.

Abuse cases are tricky. You might guess by the name that abuse cases involve only a black hat (destructive) activity. That would be wrong. Abuse cases are themselves driven by the two threads. White hat (constructive) thinking drives security requirements, which

are a necessary foundation for a goodly percentage of the abuse cases. Black hat thinking in the form of attack patterns drives the remaining portion. Though abuse cases clearly involve a mix of both hats, the predominant hat is black.

Security requirements and the resulting security functionality are squarely constructive, white hat activities. These are defined and built as an explicit defense against the black hat world. In fact, the notion of security requirements is in some sense the ultimate white hat activity.

Security operations is a white hat activity, but it is only very weakly constructive. Operations is essential to security, of course, but in terms of building security in, the tactics carried out by network-faced ops people are largely defensive.

Many of the touchpoints amount to assurance activities focused on assessing the security situation by looking at the state of various artifacts. Others, like abuse case development and security test planning, involve creating security-related artifacts from scratch. In general, those activities that involve creating new artifacts are in the business of attack creation, design, and simulation.[3] They are, in a sense, the kinds of activities best carried out with your black hat on. The others are more about constructing software properly. They are best performed while wearing your white hat.

[3] It's peculiar that these "constructive" activities—building new artifacts—are really destructive in nature! Such are the vagaries of software security.

Software security requires a matching set of both black hats and white hats, inextricably bound together.

Moving Left

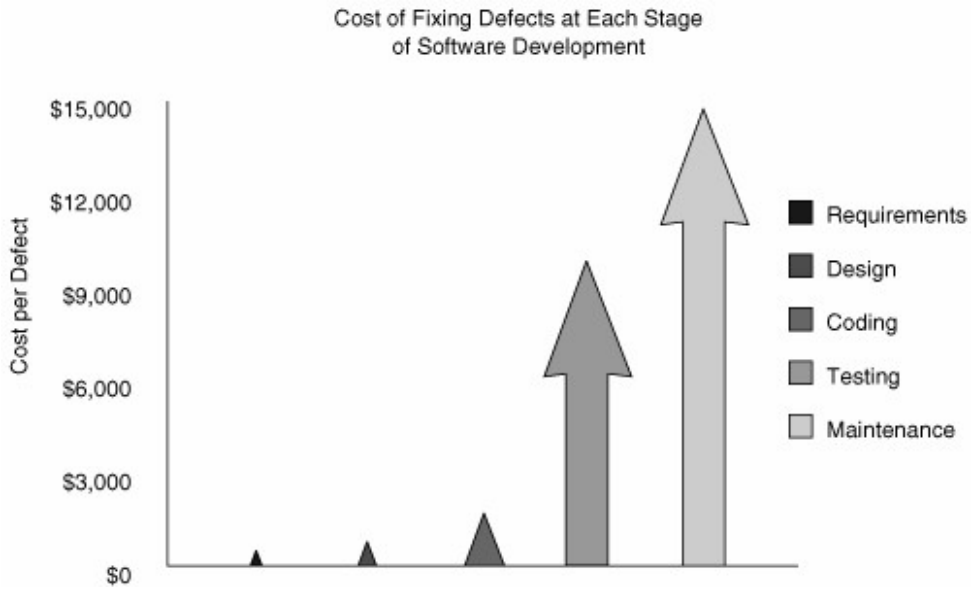
Software people know that it is much more economical to find software defects early in the lifecycle than it is to find them later. Academia provided some data about this during the 1970s but has been remiss in its duty to drive the point home with even more data.[4] Nevertheless, the fact is that fixing a problem at the requirements stage (before design, architecture, and code exist) is bound to be much cheaper than fixing even a simple bug once thousands or millions of copies of the fielded software are installed.

[4] The most oft-cited data in this regard are those gathered by TRW and IBM under the guidance of Barry Boehm <http://sunset.usc.edu/people/barry.html>. See Figure 3-2.

Simply put, early is better (Figure 3-2). This fact may seem to run at cross-purposes with the "effectiveness" ordering of the touchpoints that I suggest. However, effectiveness for me takes into account much more than simply cost. I also thought about which software artifacts are likely to be available, what kinds of tools exist (and how good they are), and the challenge presented by cultural change. When you factor in those things, I stand by my ordering.

Figure 3-2. Data from Barry Boehm's work showing how much cheaper it is to fix a defect early in the lifecycle. Use this chart to convince management of the importance of starting early. Source: TRW

[View full size image]



If early is better, it seems somewhat crazy to focus all of our attention in software security at the end of the lifecycle. But that's what we seem to be doing. Hiring reformed hackers to carry out a penetration test against your fielded software or running some kind of penetration testing tool is probably better than doing nothing. But when these late lifecycle methods find problems in your software, what are you going to do? This reactive strategy (which is really a kind of penetrate-and-patch approach) may well work OK when the fix involves something operational or environmental in nature such as installing a better operating system version, changing firewall rules, or otherwise tweaking an operational environment. But a reactive approach doesn't work so well when the problems are deep in the software itself (which is, frankly, where most of the core problems are). The state of the practice, "penetration testing first," is not very clever. One caveat is in order. Penetration testing can be very effective in lighting the security fire. That is, in a skeptical organization that thinks it is doing everything right from a security perspective, there is nothing quite as powerful as a working, demo-able remote exploit to scare the heck out of people. Use this approach with great care.

Actually, there is one strategy worse than "penetration testing first," and that is the "panic when attacked" approach. Large numbers of organizations are so far behind in computer security that they don't even realize what trouble they're in until it's way too late. If you're reading this book, you're not likely in that boat.

The answer to both of these lame strategies is to "push left" in the touchpoints diagram (Figure 3-1). In fact, the top two touchpoints—code review (with a tool) and architectural risk analysis—exist just to the left of penetration testing. In terms of economic return,

those touchpoints further to the left are going to perform better. (Of course, return alone is not the best measurement for the efficacy of a touchpoint.) In a nice coincidence, the "push left" rule gets us to the top two touchpoints very early in the game.

I predict that the software security world will soon move left into code review and that this will result in great benefit. Much more sophisticated tools exist now than were around only a few short years ago. Of course, code review with an advanced tool is no panacea for software security. We know that even the best tool in the world will find only about half the problems. Of course, finding half of the problems sure beats finding none of them.

Evidence of the move to the left already exists. A number of traditional IT firms that offered network security testing and very basic application security testing with black box tools are beginning to offer security code review (using tools, of course). This is an encouraging development.

Next will come a wave of architectural risk analysis. This is a much trickier undertaking, best performed by experts today. With better knowledge and better process models, risk analysis will be adopted by a much larger target market. In absence of in-house experts, start with your existing requirements managers and other savvy stakeholders and enhance them with outside consultants until they get on their feet. If your stakeholders know the domain well enough to hand-build a capacity plan (the performance analog of a risk analysis), they can hold the architects' feet to the fire during a more rigorous pencil-and-paper security review process.

Ultimately, pushing all the way left into requirements is our goal. By taking on security at the very beginning of the software lifecycle, we can really do the best job of building security in.

This natural evolution of adoption can easily be mirrored in any organization, from the largest to the smallest. Begin moving left as soon as possible (see Chapter 10). And by all means, get "inside" as quickly as you can. External penetration tests can help you determine how severe the problem is, but they do little to fix it.

In some cases, especially when outside consultants are involved, it is possible to combine best practices into a more holistic assessment. For example, my company, Cigital, ensures complete coverage of the software defect space by combining code review and architectural risk assessment into one service offering. Other potent combinations of touchpoints involve risk-based security testing married with penetration testing, security requirements analysis with abuse case development, code review with penetration testing, and architectural risk analysis with risk-based testing. Don't be afraid to experiment with combinations. The touchpoints are teased apart and presented separately mostly for pedagogical reasons

Touchpoints as Best Practices

As noted earlier, the software security field is a relatively new one. The first books and academic classes on the topic appeared in 2001, demonstrating how recently developers, architects, and computer scientists have started systematically studying how to build secure software. The field's recent appearance is one reason why best practices are neither widely adopted nor in some cases obvious.

The good news is that technologists and commercial vendors all acknowledge that the software security problem exists. The bad news is that we have barely begun to instantiate solutions; moreover, many proposed solutions are impotent. Not surprisingly, early commercial solutions to the software security problem tend to take an operational stance—that is, they focus on solving the software security problem through late lifecycle activities such as firewalling (at the application level), penetration testing, and patch management. Because security has tended to be operational in nature (especially in the corporate world, where IT security revolves around the proper placement and monitoring of network security apparatus), this operational tack is only natural. This leads to a bifurcation of approaches when it comes to software, into application security and software security.

The core of the problem is that building systems to be secure cannot be accomplished by using an operations mindset. Instead, we must revisit all phases of system development and make sure that security engineering is present in each of them. When it comes to software, this means taking a close look over all software artifacts. This is a far cry from black box testing.

Best practices are usually described as those practices expounded by experts and adopted by practitioners. As a group, the touchpoints vary in terms of adoption. While almost every organization worried about security makes use of penetration testing, very few venture into the murky area of abuse case development. Though I understand that the utility and rate of adoption varies among the touchpoints in this book, I am comfortable calling them all best practices.

Coder's Corner[*]

Count the problems in the following chunk of code. Use your knowledge of the touchpoints to think about what kinds of best practices might help you identify the different "levels" of problems here.

```
1 read(fd, userEntry, sizeof(userEntry));
2 comparison = memcmp(userEntry, correctPasswd,
strlen(userEntry));
3 if (comparison != 0)
4       return (BAD_PASSWORD);
```

Line 1: Return value from `read()` ignored. Always a bad sign but not directly resulting in an attack. This is the sort of bug that a fairly simple source code

analyzer can alert you to. Manual code inspection for quality issues can find these kinds of bugs as well.

Line 2: Comparing a user entry directly with the correct password. Hmm. The implication is that the system stores passwords in such a manner that they can be directly recovered in plaintext (as opposed to storing a hash). This is an architectural flaw best found during architectural risk analysis.

Line 2: `strlen()` relies on the `read()` plopping a null terminator down at the end of the buffer. No guarantee of that. A fancier source code analyzer can see the connection between the `read()` and the `strlen()`. Code review with a tool is helpful for finding bugs like this.

Line 3: The comparison succeeds if the entered password exactly matches the correct password or if the entered password is of length zero. Oops. Bye-bye password security. This kind of problem can be uncovered with good testing based on reasonable requirements and solid test planning.

There's a slightly more subtle truth at play here beyond the point problems discussed above. Although the example is contrived to pack lots of badness into a small number of lines, it's really not that unusual to find security problems clustered like this. The clueless are often gifted with the ability to be clueless in multiple dimensions simultaneously. The seven touchpoints help find all manner of cluelessness and stamp it out.

[*] This example is adapted from an interview in Slashdot by Paul Kocher. See <http://interviews.slashdot.org/interviews/03/03/27/1357236.shtml?tid=172>.

Fortunately, an organization is not required to put all touchpoints into practice to see progress on software security. Chapter 10 explains how to put together an enterprise-wide software security program and describes why adopting even only one or two of the software security touchpoints can help. Think of the touchpoints as a maturity map for your organization. The more you adopt and the more deeply you adopt, the better ... but every little bit helps.

As you adopt touchpoints in your organization, do not overlook the importance of a consistent approach to risk management. The RMF (see Chapter 2) provides a potent foundation for all touchpoints. There is little use in identifying security risks unless you intend to do something about them. Use the RMF to track progress against identified risks over time.

Who Should Do Software Security?

As it stands in many organizations, software security is nobody's job. Developers, architects, and other builders are often unaware of security and possess little in the way of software security knowledge. When their software suffers from security failure, they don't often feel responsible, arguing that security is up to the people in operations who install and operate the software they create.

A very common reaction among developers and software teams when confronted with a security problem in their system (say, during the presentation of risk analysis results) is "You can't do that! Nobody would ever do that! And even if they did, you're not supposed to do that!" Those software people who say things like that usually believe that security is IT's job and an infrastructure issue. By now you should know why that is incorrect. One key goal of the software security touchpoints is to arm software teams with enough information that these excuses never crop up. By understanding and thinking about security throughout the software development lifecycle, developers can avoid nasty surprises.

Operations people become upset when their pristine, mostly secure network is sullied by insecure software. They don't understand why software people produce such "crap," and they don't feel responsible for the ensuing security mess. They decry the pathetic state of software and wish that software developers knew more about security. In desperation, operations people grasp at security straws such as application firewalls and intrusion detection systems.

Obviously, this is not a healthy situation. When a security problem happens because of bad software, there really is nobody to hold responsible. The standard security people in operations are not really at fault (it's not their broken software), and neither are the software people (they're not security people). Organizationally, this is a textbook management problem.

In the best possible world, software security would be everybody's job. In a more realistic world, assigning responsibility and accountability to a particular group can help solve the problem.

One suggestion worth thinking about involves finding the person with the best handle on the way your whole software system works and tapping that person for software security. Ask who you turn to when something goes drastically wrong, but you don't have a clue about what is causing the problem. The jack-of-all-trades whom you turn to is your new software security person.[5]

[5] This is way too glib, of course (though it will appeal to those "builders" who are accustomed to the hero approach—"we threw a guy at that"). More mature organizations need a better-fleshed-out "who," "what," "where" framework. Different people accept different portions of the responsibility as you divide, conquer, and collaborate. See Chapter 10.

Building a Software Security Group

The world has not yet produced many software security people. That's a shame because the world certainly needs more. Fortunately, academia appears to be slowly rising to the occasion, and a number of schools are beginning to teach software security and/or security engineering courses (see the next box, Software Security in the Academy).

There is not enough time to wait for academia to produce the solution. Instead, software security people need to be developed inside existing organizations (like yours). If you want to invent some software security people in your organization, consider the following advice.

Don't start with security people

Though software security is certainly essential to addressing the computer security fiasco we find ourselves in, a standard reactive approach will fail. Network security people often don't know enough about software to make good software security people. They may know loads of stuff about how software operations work (even more in many cases than developers and architects know), but this is not what we need to solve the software security problem. Normal security practitioners almost never know anything about compilers, language frameworks, software architecture, testing, and the myriad other things necessary to be a solid software person.

Software Security in the Academy

A number of academic institutions now offer security courses very much relevant to Building Security In. The best include the following:

- University of California at Davis
- University of Virginia
- Johns Hopkins University
- Princeton University
- Purdue University (especially the CERIAS center)
- Rice University
- University of California at Berkeley
- Stanford University
- Naval Postgraduate School (a military school for graduates)
- University of Idaho
- Iowa State University
- George Washington University
- United States Military Academy at West Point

Just to highlight a couple of examples, here are some of the courses offered by these schools:

- Introduction to Computer Security (practice): UC Davis
- Computer and Information Security (theory): UC Davis
- Computer Security: An Intrusion Detection Approach: UC Davis
- Foundations of Computer and Information Security: UC Davis
- Computer Incident Detection and Response: Purdue
- Cryptography and Data Security: Purdue
- Penetration Analysis: Purdue
- Advanced Topics in Security (information assurance): Purdue

Instrumental groundbreaking work in security education has been spearheaded by Matt Bishop of UC Davis and Cynthia Irvine of the Naval Postgraduate School. Eugene Spafford of Purdue has also been extremely active in calling for better security education. For more on this important topic, see the "Education" department in IEEE Security & Privacy magazine <http://www.computer.org/security>.

Arming a normal infosec guy with a silly first-generation code scanner like ITS4 or a black box testing tool like Sanctum's Appscan rarely helps. Tools do not have enough smarts to turn network professionals into software people over night. Beware of security consultants who claim to be application security specialists when all they really know how to do is run ITS4 or Appscan and print out an incomprehensible report.

Start with software people

Security is much easier to learn about and grok than software development is. Good software people are very valuable, but software security is so important that these highly valuable people need to be repositioned. Also note that software people pay attention only to other software people, especially those with impressive scars. Don't make the mistake of putting lamers or newbies in front of a group of seasoned developers. The ensuing feeding frenzy is downright scary (if not hugely entertaining).

Identifying a responsible person or two is critical to a successful software security program (see Chapter 10). Not only is this important from an accountability perspective, but the sheer momentum that comes from a dedicated person can't be matched. If you want to adopt a new way to do code review (using a tool like Fortify), identify a champion and empower that person to get things done.

Often the most useful first person in a software security group is a risk management specialist charged with addressing software security risks that have been uncovered by outside consultants. Appointing a risk management person makes it much less likely that important results will be swept under the rug or otherwise forgotten by very busy organizations (and who is not busy these days?). The risk management specialist can be put in charge of the RMF.

Mentoring or otherwise training a new software security person may be impossible if there are no existing software security types in your organization. If that's the case, hire outside consultants to come and help you boot up a group. The extensive experience and knowledge that software security consultants have today are as valuable as they are rare, but it is well worth investing in mentoring your people in order to build that capability.

Ultimately, you want two types of people to populate your software security group: black hat thinkers and white hat thinkers. If you're lucky, you'll find people who can switch hats at, um, the drop of a hat. But more likely, you'll have some good constructive types (who naturally swing toward the white hat side) and some devious destructive types (who naturally swing toward the black hat side). In some sense, this matches the distinction between builders and auditors. You need both, of course, because the touchpoints demand both. Know that the builders are much more important than the auditors, though

Software Security Is a Multidisciplinary Effort

Software security as a discipline is a new undertaking. On the plus side, new disciplines benefit from a creative mix of seemingly unrelated disciplines (see the box Creativity in a New Discipline). On the negative side, software security is so new that sometimes it is not clear exactly how it should be practiced.

Software security can and should borrow from other disciplines in computer science and software engineering when developing and evolving best practices. A quick shout out to related fields is important, as the literature defining software security remains fairly sparse. The following topics are of particular relevance and well worth diving into:

- Security requirements engineering
- Design for security, software architecture, and architectural analysis
- Security analysis, security testing, and use of the Common Criteria
- Guiding principles for software security and case studies in design and analysis
- Auditing software for implementation risks, architectural risks, automated tools, and technology developments (code scanning, information flow, and so on)
- Common implementation risks (buffer overflows, race conditions, randomness, authentication systems, access control, applied cryptography, and trust management)

A number of these topics have some coverage in the annotated bibliography found in Chapter 13. Much work remains to be done in each of the best practice areas defined by the touchpoints, but other basic practical solutions should be adapted from areas of more mature practice as well.

Creativity in a New Discipline[*]

We are experiencing a time of great creativity in computer security and must seize the opportunity presented by our current situation while we can. The

diversity of backgrounds represented by today's security practitioners may be a high-water mark. Consider that today's security thought leaders were trained in fields as diverse as bio-statistics, divinity, economics, and cognitive science, and thus bring with them interesting new perspectives on the security challenge. This leads to creative interplay in the field and has resulted in interesting progress, including the emergence of economic theories of security, an embrace of risk management, an emphasis on process-driven approaches (versus product sets), a shift toward software security, the rise of security engineering, and so on. As the worldwide security paradigm shift from guns, dogs, and concrete to networks, information systems, and computers continues unabated, we must leverage this time of creative diversity for all it's worth.

A number of young researchers joined the computer security field in the mid-1990s, changing the focus of security research from spookware and national defense (think crypto, multilevel security, communications monitoring, and the like) to commercial systems and commerce. This movement away from military-oriented research was driven in part by the widespread public adoption of the Internet and the growing trend of e-commerce. With money at stake, security quickly became as relevant to business as it was to national defense. This influx of "new blood" shook up the scientific security research community and continues to have far-reaching effects that are only now affecting commercial security—the commercialization of firewalls, the rise of antivirus technology, and the adoption of modern security platforms, such as Java and .NET, were all predicted and spearheaded by new thinkers in the security research community.

Where Today's Security People Come From

Only a handful of people working in computer security today started their careers in the field. In fact, academic programs expressly designed to train security practitioners are a recent phenomenon and remain rare.

Interestingly, it may be in this dearth of "qualified" people trained in security that a critical opportunity can be found. Though few practitioners have academic security training, they most assuredly do have academic training in some field of study. That means that as a collective, the computer security field is filled with diverse and interesting points of view. This is exactly the sort of Petri dish of ideas that led to the Renaissance at the end of the Dark Ages.

Diversity of ideas is healthy, and it lends a creativity and drive to the security field that we must take advantage of. A great example of this can be found in the new subfield of software security. Only five years ago the notion that bad software might be a major root cause of security issues was not common. Today, software security is the subject of keynote talks at the RSA security conference <http://rsaconference.com/>, and we all seem to agree that we have a software problem to solve. This change was partially due to the involvement of

programming languages people (once found only at obscure academic conferences like OOPSLA) in the security field. Such involvement resulted in the creation of modern languages like Java and .NET that include security models in their very design. When languages are declared "secure," things get interesting! The evolutionary arms race between attackers and defenders jumps a level, new avenues for security design emerge, and dusty but thorny problems (think "buffer overflow") become less relevant to the next generation of systems.

Where Tomorrow's Security People Will Come From

These days, academic and professional training programs are being put in place to train the next generation of security professionals. Soon, standard curricula will be developed, and students will be required to understand the same core set of concepts. This will certainly help to solidify the field of computer security, but at the same time, there is a danger that generalization may lead to a homogenization of security. Instead of the creative soup afforded by a multiplicity of points of view spanning many fields, security runs the risk of becoming staid and static. If we are careful to avoid complete homogenization of the field, we can retain the benefits of diversity while building a solid academic discipline. One way to do this might be to encourage those students seeking computer security degrees to study widely in other supposedly unrelated disciplines as well. Another is to ensure that outside perspectives remain welcome in the field and are not dismissed out of hand. Computer security must remain an inclusive discipline in order to retain its creativity.

In any case, we must take advantage of the situation we find ourselves in now. Computer security is, in fact, experiencing an important rebirth, and now is the time to make great progress. We must pay close attention to different ideas, embrace change, and help security continue to evolve even as it begins to crystallize.

[*] Portions of this text box originally appeared in my Network magazine "[In]security" column from February 2005 entitled "Are We in a Computer Security Renaissance?" The seed idea came from a conversation with Dan Geer. Network magazine is now IT Architect.

Touchpoints to Success

As I have said before, software security is not security software. Security functionality alone will not make software secure. The touchpoints outlined here reinforce and flesh out that perspective by emphasizing the kinds of assurance activities necessary to build security in. To attain software security, software projects must apply the touchpoints throughout the software lifecycle, practicing security assurance as they go. The

touchpoints I have identified take into account both security mechanisms (such as access control) and design for security (such as robust design that makes software attacks difficult). These encompass both black hat and white hat activities. Sometimes the areas overlap, but often they don't. They are, however, closely aligned.

One central goal of this book is to describe the best practices overviewed in this chapter in more detail. Touchpoints are one of the three pillars of software security. As the connectedness, complexity, and extensibility of modern software continue to impact software security in a negative way, we must begin to grapple with the problem in a more reasonable fashion than simply spray painting cryptography on our code. Integrating a decent set of best practices into the software development lifecycle is an excellent way to do this. Playing the game of software security requires both good offense and good defense (in other words, two hats), and for that reason the touchpoints use both constructive and destructive approaches. Although software security as a field has much maturing to do, it already has a lot to offer to those practitioners interested in striking at the heart of security problems.